

Tuan Anh Nguyen

Senior Unity Developer | Mobile Game Developer | Performance & LiveOps

Hanoi, Vietnam | talama191@gmail.com | +84 838 331 032

linkedin.com/in/tuan-anh-nguyen-31648513a | github.com/talama191

SUMMARY

Senior Unity / C# developer with 6 years shipping and running live mobile titles at scale, including *Lumber Inc* (100M+ downloads on Google Play), where I'm second-in-command on the project, mentoring 3 developers. Strongest on performance: profiling-led CPU, memory and draw-call optimization across device tiers, plus rewriting systems a live game has outgrown. Examples: a 3D storage system held under ~10 KB warm-up across thousands of objects, ~43% load-time reduction on low-end Android. Delivers live-ops features end to end, architected as true A/B tests rather than just tuning numbers, and builds the tooling, automated testing and CI/CD pipeline behind them, AI-agent-assisted (Claude Code, Unity MCP) included. Open to relocation.

SKILLS

Languages: C#, TypeScript, Java, SQL, JavaScript

Engineering: OOP and design patterns (strategy-based variants, data-driven ScriptableObject architecture), data structures and algorithms on hot paths (3D packing cache, spatial-grid broad phase, runtime mesh triangulation), refactoring systems a live game has outgrown, unit and scenario-based automated tests

Performance: CPU & memory profiling, allocation/GC reduction, draw-call reduction (GPU instancing, texture batching, atlasing, canvas splitting), load-time optimization

Unity: UGUI / Canvas, Event System, Graphic Raycaster, DOTween, Animator, Addressables, UniTask, HLSL shaders, Editor scripting, Unity Profiler, Unity Test Framework

Mobile & LiveOps: iOS, Android, in-app purchases (IAP), limited-time events, A/B testing, Firebase (Remote Config, Analytics, Crashlytics), Google Play Games Services, Beamable (client SDK), ad-network SDKs

Build & CI: Git, GitLab CI/CD and runners (Windows, macOS), headless Unity (batchmode) builds, Bash, xcodebuild, Android and iOS release signing, TestFlight upload

AI-Assisted Development: Claude Code, Unity MCP, used for scripting, internal tooling and test authoring; own architecture, code quality and review regardless of source

WORK EXPERIENCE

Senior Game Developer

Nopowerup | Hanoi, Vietnam | Dec 2022 - Present

Mobile simulation and tycoon games in Unity and C#. On Lumber Inc since Aug 2024.

Lumber Inc: idle tycoon, publisher AppQuantum, 100M+ downloads on Google Play
play.google.com/store/apps/details?id=com.lumber.inc (not available on Vietnam store)

- Second-in-command on the project since the prior lead moved on: mentor 3 developers, decide most day-to-day technical questions and escalate the genuinely hard ones. One of two regular code reviewers alongside the tech lead; I onboard new joiners with a codebase walkthrough and first-task pairing.
- Client-side feature delivery on a live build: the shop, limited-time events, IAP offers and the piggy-bank system, each with the UI it carries. Wired up Remote Config so live-ops can switch content on and retune rewards and economy without a new release.
- Built the forklift coordination system matching storage contents to incoming orders: 3 order-fulfillment and several logistics-routing strategies coexist as true A/B tests, selected by Remote Config and swapped on restart, with losing variants cut once the data is in.
- Built the UI framework the game's screens sit on, including a redirect system: given a destination, it closes whatever popups are open, switches to the correct tab and scrolls the target into view, so a promotion lands on the exact thing being promoted rather than on a menu.
- Rewrote package storage when the publisher needed non-uniform product shapes the shipped 2D row/column layout could not represent: true 3D stacking plus a caching strategy that holds warm-up under ~10 KB allocation and a few ms per query across thousands of boxes and 40-50 storage areas.
- Built Google Play Games cloud save, so players who never linked an account get their progress back after a reinstall or on a new phone. When a device save and a Play Games save both exist, the player compares them and picks, so neither is overwritten silently.
- Profiling-led CPU and memory passes on the simulation loop. Draw-call reduction via GPU instancing and texture batching, plus canvas splitting, redraw batching and atlasing where the UI itself was the cost.

- Built the project's GitLab CI/CD pipeline, replacing a deprecated one: headless Unity jobs on Windows and Mac runners produce Android and iOS debug, signed release and low-end asset-variant builds, push Addressables bundles to the content server and upload iOS builds to TestFlight.
- Built a scenario-based automated test framework used by the test team: testers compose scenarios from steps (set simulation stage, set currency, trigger an unlock, open a popup, assert a state) and add their own scripts, extending coverage without an engineer in the loop. Unit tests on the systems that warrant them.
- Built the tooling around the game: a tap-triggered on-device debug menu for testers, a Google-Sheets-to-Unity balance export pipeline for designers, an Addressables duplication checker that stops assets silently duplicating into the base build, and a Claude Code-assisted event-shell duplicator I use to set up new limited-time events faster.
- Triage live crashes from Firebase Crashlytics. First port of call from QA, designers and artists when something breaks, with a confirmed turnaround on the spot.
- Wrote small HLSL effects, for example lumber dissolving as it enters a machine. Traced an iOS-only rendering bug to a shader using half4, where the lower precision surfaced as pixel artifacts on device.

Milk Farm Tycoon: idle tycoon, publisher Eastside Games

play.google.com/store/apps/details?id=com.eastsidegames.milking

- Built the conveyor and distribution simulation. Built the battle pass, the transporter-renting system (players rent trucks to deliver customer orders) and in-game localization against the Beamable client SDK, so progression, rental state and economy data persist server-side rather than in a local save.
- CPU optimization on the simulation loop via targeted profiling passes, draw-call reduction via GPU instancing and texture batching, and a fake-shadow-mesh tool for the team.

Puzzle prototype

- Prototyped a puzzle game in the style of *Where's My Perry?*, with sand in place of the liquid: player swipes carve away sand so balls can fall through the cuts, and the sand mesh is rebuilt at runtime by triangulation.

Unity Instructor (Part-time)

CodeGym | Hanoi, Vietnam | Aug 2023 - Aug 2024

- Taught two cohorts of aspiring developers Unity and C# game development, from programming fundamentals to their own playable projects, reviewing their work and mentoring one-on-one.

Game Developer

Rainbow Studio | Hanoi, Vietnam | Aug 2020 - Aug 2022

Mobile hyper-casual games (Unity, C#) and web games (Egret engine, TypeScript).

- Gameplay and UI implementation across multiple small titles, ad-network and IAP SDK integration, and internal tools for processing raw sprite, animation and game data.

PROJECTS

Sport Stars, Unity mobile collection RPG in a sports setting

play.google.com/store/apps/details?id=com.bruhs.sportstars

A self-directed, co-developed, shipped side project, in development since Aug 2023: roughly 1,800 commits, the second-largest contributor on the codebase. Players collect and level character cards carrying rolled abilities and per-discipline stats, build squads and push campaign and ladder content.

- Built the RPG layer: the character stats-and-abilities system, where rarity-tagged ability assets roll onto a card and resolve into per-match runtime behaviours, and the real-time match simulation those stats and abilities drive across eight disciplines.
- Built the HQ management sim where players train those characters: facility logistics with an autonomous agent AI that routes each card through a designer-authored routine, the amenity system, and the training assignment that grows each character's stats. Also built the mission and reward system.
- Reduced load time on low-end Android (Samsung A07-class hardware) from ~35 s to ~20 s, roughly 43% on the device class where load time hurts most. CPU optimization across the update loop.

EDUCATION

Bachelor of Science in Computer Science (Software Engineering)

Hanoi University | Hanoi, Vietnam | Aug 2018 - Jun 2023

LANGUAGES

English (C1, professional working proficiency), Vietnamese (native)